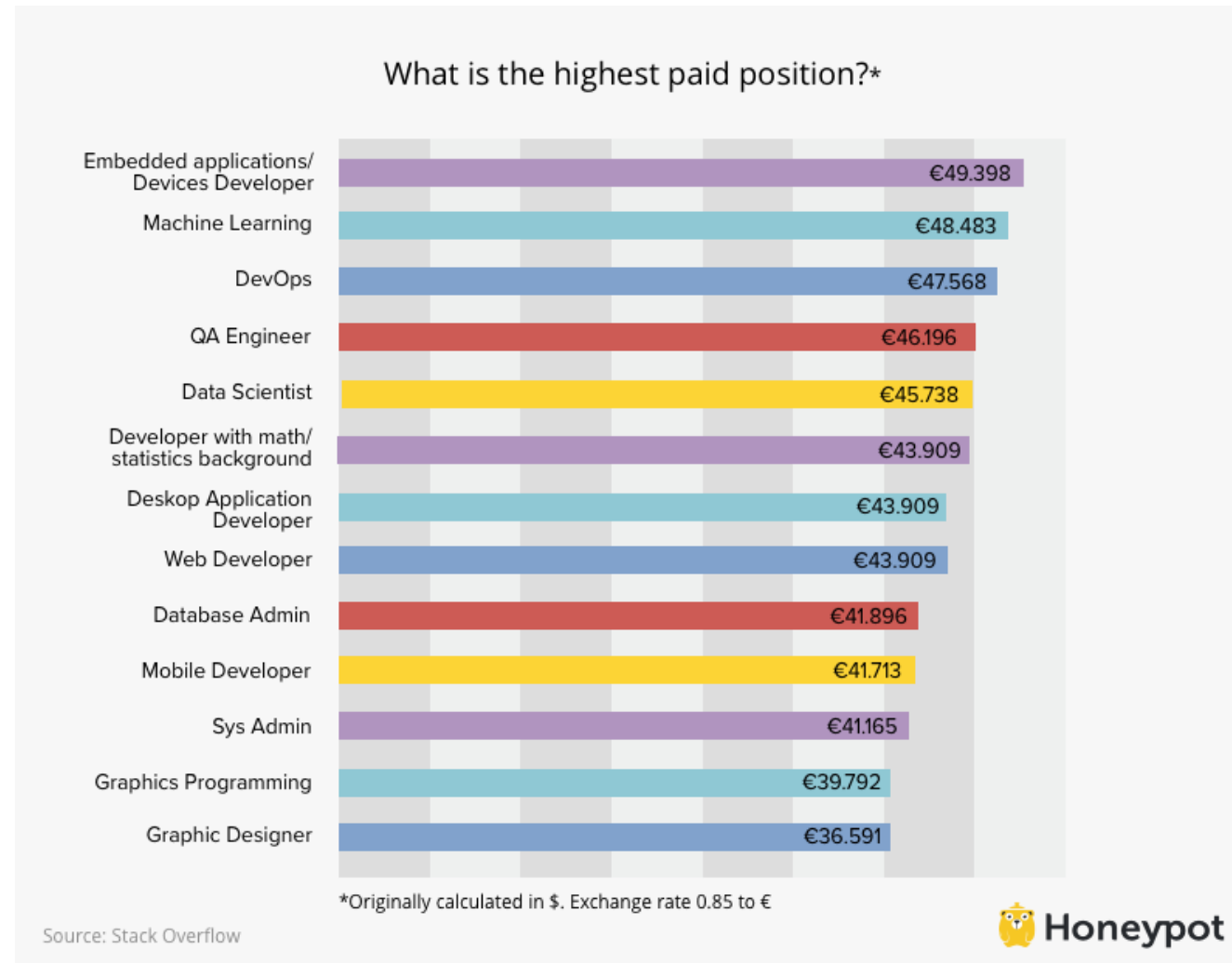


MACHINE LEARNING

Polynomial regression

1

CODE EXAMPLE — POSITION SALARIES



LOAD DATASET

Importing the libraries

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

Importing the dataset

```
dataset = pd.read_csv('Position_Salaries.csv')
```

Index	Position	Level	Salary
0	Business Analyst	1	45000
1	Junior Consultant	2	50000
2	Senior Consultant	3	60000
3	Manager	4	80000
4	Country Manager	5	110000
5	Region Manager	6	150000
6	Partner	7	200000
7	Senior Partner	8	300000
8	C-level	9	500000
9	CEO	10	1000000

Format

Resize



Background color



Column min/max

OK

Cancel

DEPENDENT & INDEPENDENT VARIABLES

```
X = dataset.iloc[:, 1:2].values
```

```
y = dataset.iloc[:, 2].values
```

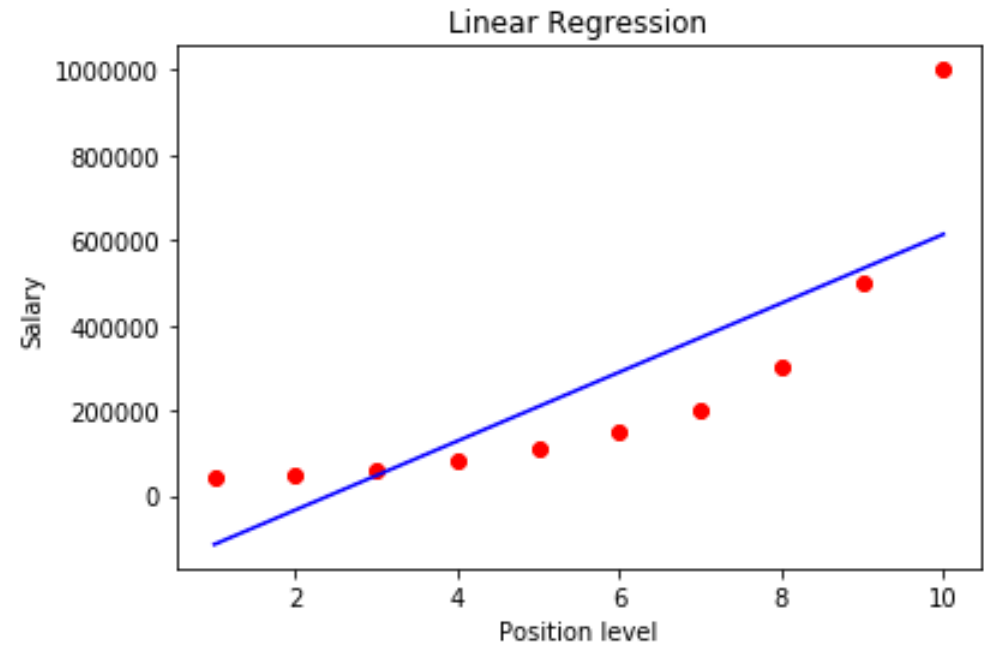
TRAINING AND TESTING SETS

Not enough Data....

FITTING LINEAR REGRESSION TO THE DATA

```
from sklearn.linear_model import LinearRegression  
lin_reg = LinearRegression()  
lin_reg.fit(X, y)  
#we want to compare Linear regression model to polynomial regression
```

```
plt.scatter(X, y, color = 'red')  
plt.plot(X, lin_reg.predict(X), color = 'blue')  
plt.title('Linear Regression')  
plt.xlabel('Position level')  
plt.ylabel('Salary')  
plt.show()
```



VISUALIZING THE LINEAR REGRESSION RESULTS

FITTING POLYNOMIAL REGRESSION

#1. We need to change X to have more independent variables:

```
from sklearn.preprocessing import PolynomialFeatures
```

```
poly_reg = PolynomialFeatures(degree = 2)
```

```
X_poly = poly_reg.fit_transform(X)
```

#2. Now we can use linear regression model:

```
lin_reg_2 = LinearRegression()
```

```
lin_reg_2.fit(X_poly, y)
```

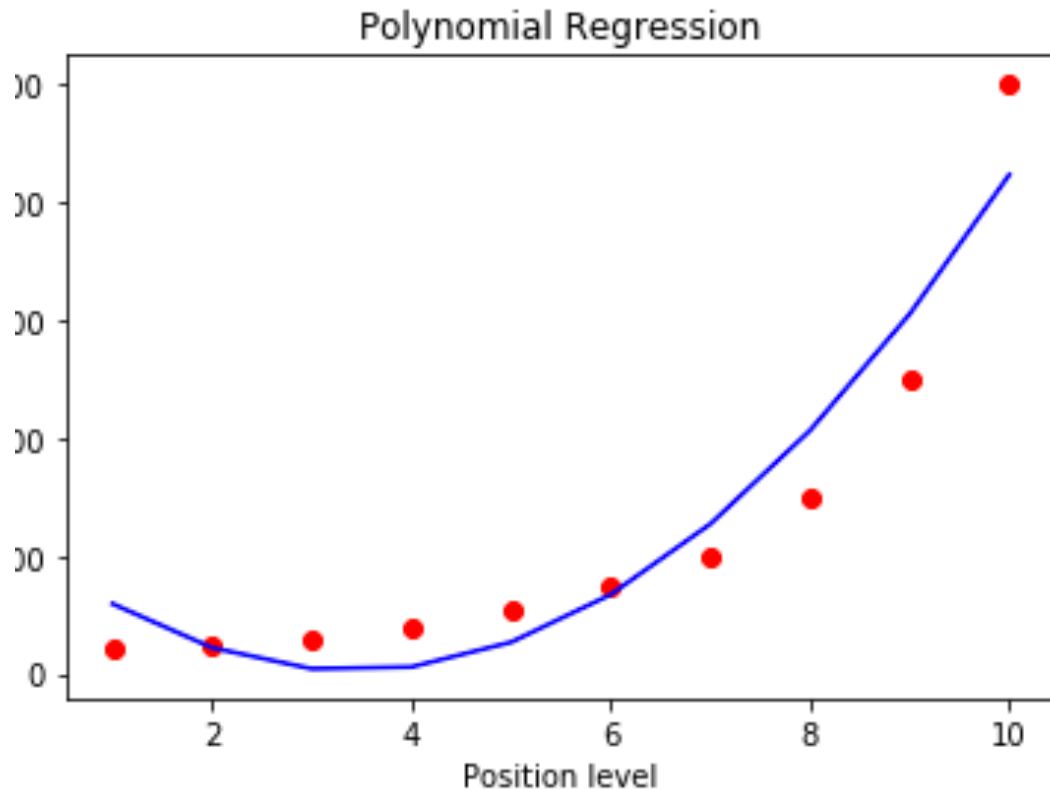
X_poly - NumPy array

	0	1	2
0	1	1	1
1	1	2	4
2	1	3	9
3	1	4	16
4	1	5	25
5	1	6	36
6	1	7	49
7	1	8	64
8	1	9	81
9	1	10	100

Format Resize ☒ Background color

OK Cancel

VISUALIZING THE POLYNOMIAL REGRESSION RESULTS

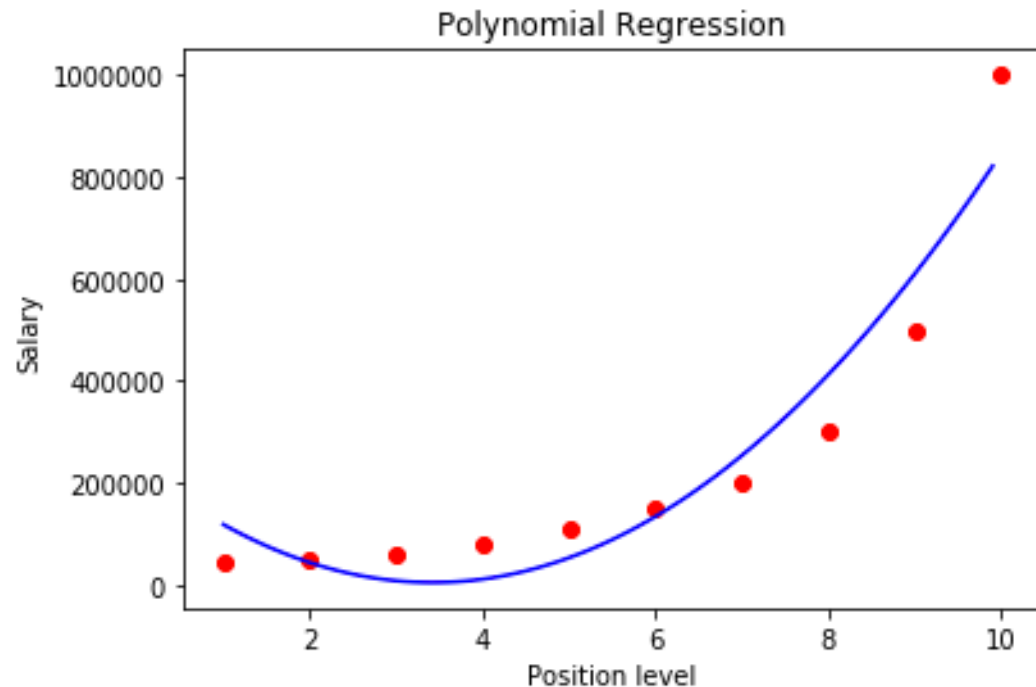


```
plt.scatter(X, y, color = 'red')  
  
plt.plot(X,  
lin_reg_2.predict(poly_reg.fit_transform(X))  
, color = 'blue')  
  
plt.title('Polynomial Regression')  
  
plt.xlabel('Position level')  
  
plt.ylabel('Salary')  
  
plt.show()
```

#Nice, but can be improved.

SMOOTHER CURVE

```
X_grid = np.arange(min(X), max(X), 0.1)    #we get a vector  
X_grid = X_grid.reshape((len(X_grid), -1)) #we need a matrix, have single sample  
plt.scatter(X, y, color = 'red')  
plt.plot(X_grid, lin_reg_2.predict(poly_reg.fit_transform(X_grid)), color = 'blue')  
plt.title('Polynomial Regression')  
plt.xlabel('Position level')  
plt.ylabel('Salary')  
plt.show()
```



CROSS VALIDATION

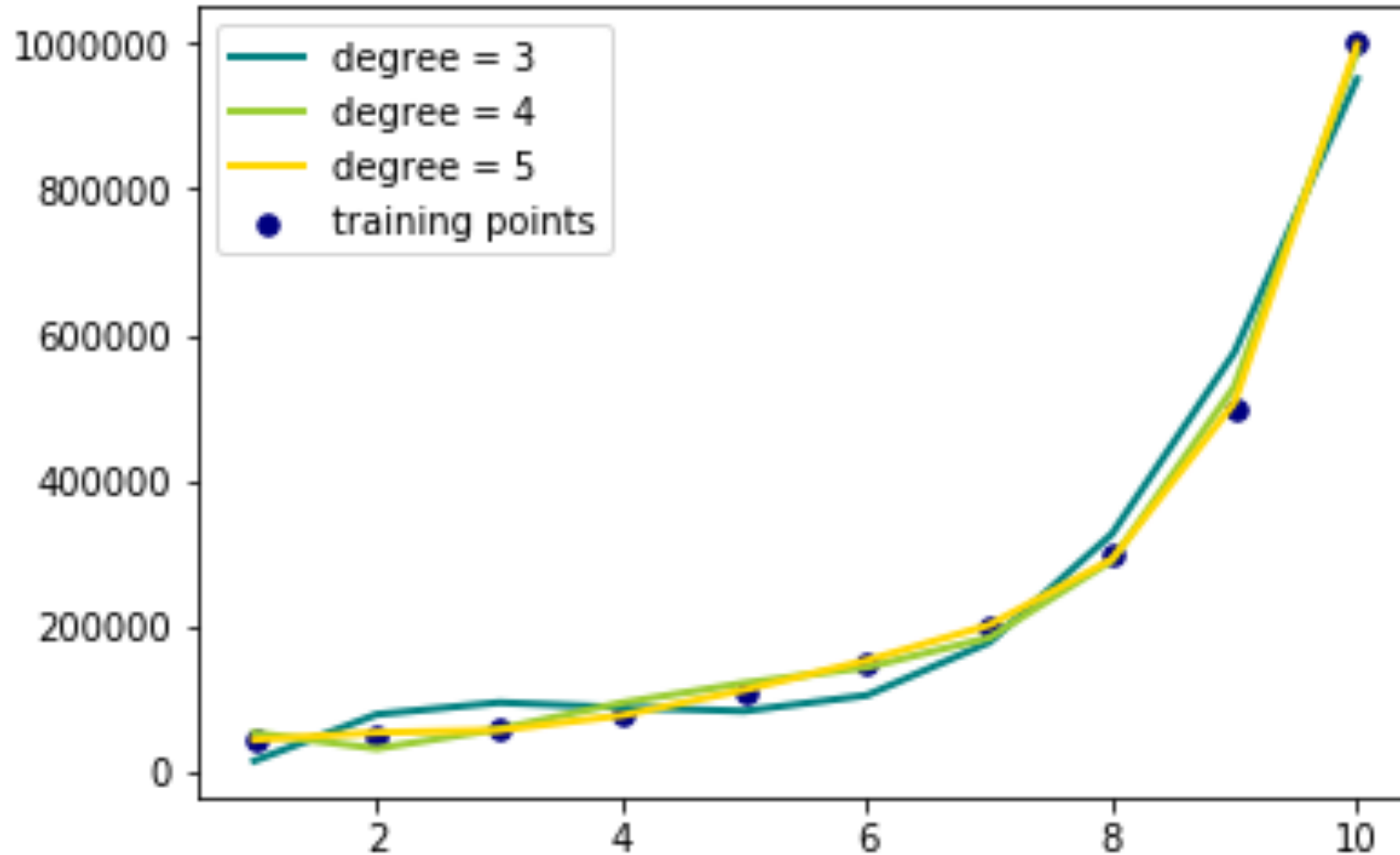
```
from sklearn.model_selection import cross_val_score

accuracies=cross_val_score(estimator=lin_reg2,X=X_poly,y=y,cv=10,scoring='neg_
mean_squared_error') #you can also try scoring='r2'

print(accuracies.mean())

print(accuracies.std())
```

EVALUATING MULTIPLE DEGREES...



EVALUATION

```
from sklearn.metrics import mean_squared_error, r2_score  
rmse = np.sqrt(mean_squared_error(y,y_poly_pred))  
r2 = r2_score(y,y_poly_pred)
```

Polynomial Regression Results:

Degree	RMSE	R²
1	163388.74	0.67
2	82212.12	0.92
3	38931.50	0.98
4	14503.23	1.00
5	4047.50	1.00